

IILM UNIVERSITY, GREATER NOIDA

School of Computer Science and Engineering

AN INTERNSHIP REPORT ON PYTHON DEVELOPER INTERNSHIP

Internship Project

API Performance Monitor

Submitted in partial fulfilment of the requirements for
B.Tech (Computer Science and Engineering)



Submitted By

ROHAN PATIL

Roll No.: 25SCS1003005701

Internship Role: Python Developer Intern

Organization: Codec Technologies Pvt. Ltd.

Duration: 25 July 2026 – 25 August 2026

CANDIDATE'S DECLARATION

I, ROHAN PATIL, hereby declare that the internship report titled “Python Developer Internship – API Performance Monitor” has been prepared by me as a record of my internship work at Codec Technologies Pvt. Ltd. The report presents the learning, development activities, implementation approach, and outcomes of the internship and project work.

I affirm that the work described in this report has been prepared for academic submission and that the technical implementation and observations presented here are intended to accurately represent the internship project completed during the stated internship period.

I understand that I am responsible for the authenticity and appropriate use of the material included in this report.

Student Name: ROHAN PATIL

Roll No.: 25SCS1003005701

Date: 2026

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to Codec Technologies Pvt. Ltd. for providing me the opportunity to undertake the 1 Month Python Developer Internship and gain practical exposure to Python programming, web development, data handling, APIs, debugging, and project implementation.

I am thankful for the structured learning experience that helped me strengthen my understanding of Python fundamentals and advanced concepts and apply them to a practical software project. The internship provided an opportunity to move from learning individual concepts to integrating them into a working application.

I would also like to thank IILM University, Greater Noida, and the faculty of the School of Computer Science and Engineering for their academic support and encouragement.

Finally, I am grateful to my family and everyone who supported me throughout the internship and encouraged me to complete the project successfully.

ROHAN PATIL

Roll No.: 25SCS1003005701

TABLE OF CONTENTS

1. Candidate's Declaration	i
2. Acknowledgement	ii
3. Internship Completion Certificate	-
4. Project Description	1
4.1 Introduction	1
4.2 Organization & Internship Profile.....	2
4.3 Problem Statement.....	3
4.4 Project Objectives.....	3
4.5 Scope of the Project	4
4.6 Technologies and Tools Used.....	4
4.7 System Architecture.....	5
4.8 Methodology.....	6
4.9 Implementation & Key Features	7
4.10 Results and Project Output.....	7
4.11 Learning Outcomes.....	8
4.12 Conclusion.....	9
5. Bibliography/References.....	10

3. INTERNSHIP COMPLETION CERTIFICATE

The internship completion certificate issued by Codec Technologies Pvt. Ltd. confirms successful completion of the 1 Month AICTE & ICAC Approved internship program in the role of Python Developer Intern. The certificate records the internship period as 25/07/2026 to 25/08/2026 and displays the AICTE ID CORPORATE6759d549ce59e1733940553.



4. PROJECT DESCRIPTION

4.1 Introduction

This report presents the work completed during the 1 Month Python Developer Internship at Codec Technologies Pvt. Ltd. The internship focused on developing practical proficiency in Python and related technologies, including Flask, HTTP/API interaction, data storage, file handling, debugging, and application development.

As the project component of the internship, an API Performance Monitor was developed. The application provides a simple web interface through which a user can enter an HTTP/HTTPS endpoint, send a request, measure response time, inspect the HTTP status code, record whether the request succeeded or failed, and view recent monitoring results.

The project demonstrates how Python programming concepts can be combined with a web framework, an HTTP client library, and a lightweight database to create a useful monitoring application.

4.2 Organization & Internship Profile

Codec Technologies Pvt. Ltd. served as the organization for the internship. The certificate issued by the organization identifies the role as Python Developer Intern and records the internship duration from 25 July 2026 to 25 August 2026.

The internship learning path covered Python fundamentals and data structures, advanced Python and object-oriented programming, file and structured-data handling, Flask-based web development, data handling, REST APIs, requests, debugging, and a project phase. The project phase was used to consolidate these skills in a practical application.

4.3 Problem Statement & Project Objectives

4.3 Problem Statement

Modern software systems frequently depend on APIs to exchange data between applications and services. When an API becomes slow, unavailable, or returns an unexpected HTTP status, it can affect the performance and reliability of applications that depend on it. Manually checking endpoints does not provide a convenient record of response times and historical results.

The problem addressed by this project is to provide a simple monitoring interface that can test API endpoints, measure their response performance, identify successful and failed responses, and maintain a local record of checks for quick analysis.

4.4 Project Objectives

- Develop a Python-based web application for testing HTTP/HTTPS API endpoints.
- Measure and display API response time in milliseconds.
- Capture HTTP status codes and classify requests as successful or failed.
- Store monitoring results in a SQLite database for later viewing.
- Provide dashboard-level metrics such as request count, average response time, and success rate.
- Apply Python, Flask, Requests, SQLite, HTML, CSS, and JavaScript concepts in one working project.
- Create a project that can be demonstrated locally and included as practical evidence of internship learning.

4.5 Scope of the Project

The scope of the API Performance Monitor is centered on lightweight API testing and local performance logging. The system is designed as a demonstration and learning project rather than a production-grade distributed monitoring platform.

- Testing user-supplied GET endpoints over HTTP/HTTPS.
- Measuring elapsed response time for each request.
- Recording HTTP status codes and success/failure results.
- Persisting checks locally using SQLite.
- Displaying recent checks through a browser-based Flask dashboard.
- Calculating summary statistics from recorded requests.
- Providing a clear foundation that can be extended with scheduled monitoring, charts, authentication, alerts, and multi-user support.

Functional Requirements

- The user can enter an API endpoint.
- The application sends the request and measures elapsed time.
- The result is stored and displayed in the dashboard.
- The dashboard shows request totals, successful/failed counts, average response, and success rate.
- The application provides a clear-log function.

4.6 Technologies and Tools Used

Technology / Tool	Purpose
Python	Core programming language and application logic
Flask	Web framework for routes, requests, and rendering templates
Requests	Sending HTTP requests to external API endpoints
SQLite	Lightweight local database for storing monitoring logs
HTML	Structure of the web dashboard
CSS	Dashboard layout and visual styling
JavaScript	Client-side interaction and dashboard updates
VS Code / IDE	Writing, running, and debugging the application

The project intentionally uses a lightweight technology stack so that it can be run locally without requiring a large infrastructure setup. Flask handles the application layer, Requests handles HTTP communication, and SQLite provides persistent local storage.

4.7 System Architecture

The application follows a simple layered workflow. A user interacts with the Flask web interface, the server validates the endpoint and performs the HTTP request, the result is measured and stored in SQLite, and the dashboard displays both the individual check and aggregate statistics.

Layer	Function
User / Browser	Enters API URL and views dashboard
Flask Web Layer	Receives the request and coordinates the monitoring operation
Requests Layer	Sends HTTP GET request to the target endpoint
Measurement Layer	Records elapsed time and HTTP status
SQLite Database	Stores URL, status, result, response time and timestamp
Dashboard	Displays recent checks and summary metrics

User → Flask → Requests → API Endpoint



Response Time + Status



SQLite Database



Monitoring Dashboard

4.8 Methodology

The project was developed using an iterative workflow in which the application was built, executed locally, tested with multiple endpoints, and refined when errors were encountered.

Stage	Work Performed
1. Requirement Analysis	Defined the purpose of the monitor, dashboard metrics, request logging, and local database storage.
2. Environment Setup	Created a Python virtual environment and installed the required packages from requirements.txt.
3. Backend Development	Implemented Flask routes, API request handling, response-time measurement, and SQLite operations.
4. Frontend Development	Created a dashboard for entering endpoints and viewing summary statistics and recent checks.
5. Testing	Tested successful endpoints, a 404 endpoint, and a delayed endpoint to verify different outcomes.
6. Debugging	Resolved runtime issues and validated that successful and failed requests were recorded correctly.
7. Documentation	Prepared project documentation and captured the working dashboard as evidence for the internship report and presentation.

4.9 Implementation & Key Features

Implementation Overview

The Flask application exposes a web interface for entering a target endpoint. When the user selects Check API, the backend sends a GET request using the Requests library and measures the elapsed time. The HTTP status code is used to determine whether the check is successful. The application stores the result in SQLite and returns updated dashboard information to the browser.

Key Features

- Endpoint testing through a browser-based form.
- Response-time measurement in milliseconds.
- HTTP status-code tracking.
- Success/failure classification.
- Persistent SQLite logging.
- Request count, success count, failure count, average response, and success-rate metrics.
- Fastest and slowest response-time summary.
- Recent API checks table with timestamp and result.
- Clear Logs functionality for resetting the local monitoring history.

Example Test Cases

Endpoint	Expected / Observed	Purpose
https://jsonplaceholder.typicode.com/todos/1	200 / SUCCESS	Normal JSON API test
https://httpbin.org/get	200 / SUCCESS	GET request test
https://httpbin.org/status/200	200 / SUCCESS	Successful status test
https://httpbin.org/status/404	404 / FAILED	Failure-status test
https://httpbin.org/delay/2	200 / SUCCESS	Slow-response test

4.10 Results and Project Output

The completed application was executed locally and tested with five API requests. The dashboard recorded four successful requests and one failed request. The failed request was the 404 status test, while the delayed endpoint produced the slowest response time.

Metric	Observed Result
Total Requests	5
Successful Requests	4
Failed Requests	1
Success Rate	80.0%
Average Response	1510.9 ms
Fastest Response	436.63 ms
Slowest Response	3267.13 ms

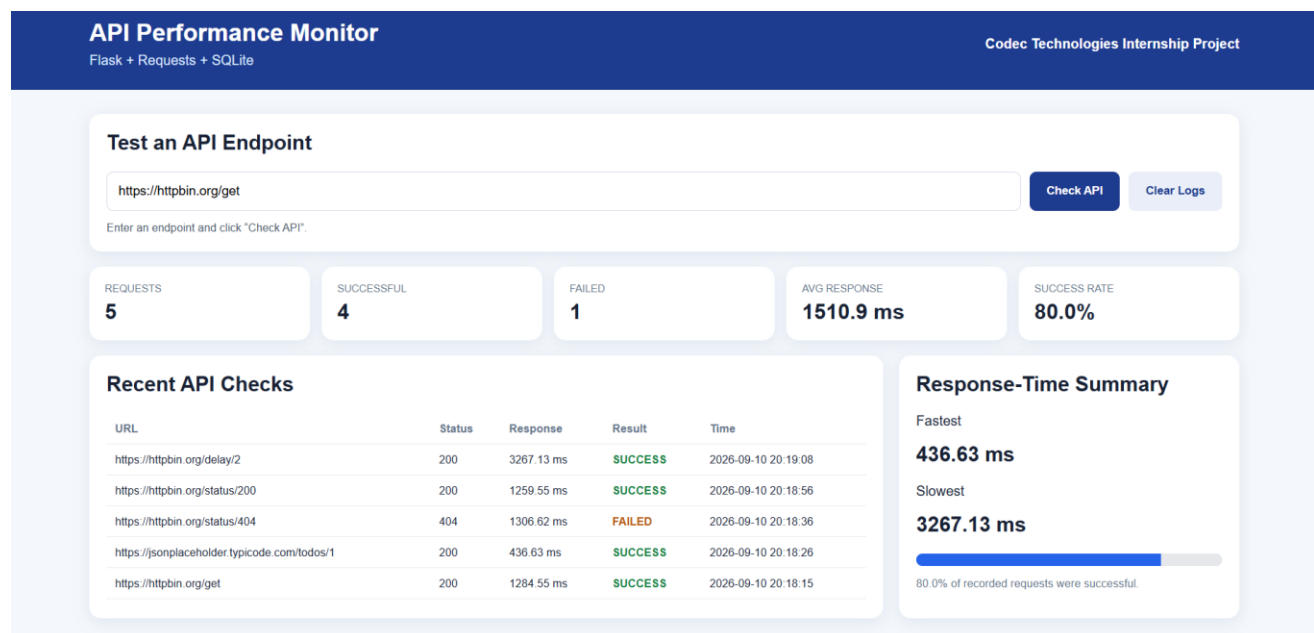


Figure: Working API Performance Monitor dashboard showing recorded API checks and response-time summary.

4.11 Learning Outcomes

The internship and project helped translate Python concepts into a complete working application. The major learning outcomes are summarized below.

- Improved understanding of Python syntax, variables, data structures, functions, exceptions, and modular programming.
- Practical use of object-oriented and structured programming concepts.
- Experience with Flask routes, request handling, templates, and web application structure.
- Understanding of HTTP requests, status codes, API endpoints, and response-time measurement.
- Practical use of SQLite for persistent application data.
- Experience with debugging and resolving runtime/application issues.
- Understanding of how backend logic and frontend dashboard components work together.
- Experience in testing an application using different success, failure, and slow-response scenarios.
- Improved ability to document and present a software project.

Professional Development

The internship also strengthened problem-solving, debugging, documentation, and project-presentation skills. Building a complete application from setup through testing provided practical exposure to the software development workflow.

4.12 Conclusion

The 1 Month Python Developer Internship at Codec Technologies Pvt. Ltd. provided a structured opportunity to develop and apply Python programming skills in a practical setting. The internship covered core programming concepts, advanced Python topics, web development, data and API handling, and project implementation.

The API Performance Monitor served as the practical project for integrating these skills. The completed application successfully tested API endpoints, measured response times, captured HTTP status codes, stored results in SQLite, and presented useful monitoring metrics through a Flask dashboard.

The project demonstrates a complete development cycle from requirement definition and environment setup to implementation, testing, debugging, and documentation. It also provides a foundation for future improvements such as scheduled monitoring, historical charts, alerts, authentication, and deployment.

— END OF REPORT —

5. BIBLIOGRAPHY / REFERENCES

1. Codec Technologies Pvt. Ltd., “1-Month Python Developer Internship,” course curriculum and internship learning material, 2026.
2. Codec Technologies Pvt. Ltd., “Certificate of Internship,” Python Developer Intern, 25/07/2026–25/08/2026.
3. Python Software Foundation, Python Documentation, <https://docs.python.org/3/>
4. Flask Documentation, <https://flask.palletsprojects.com/>
5. Requests Documentation, <https://requests.readthedocs.io/>
6. SQLite Documentation, <https://www.sqlite.org/docs.html>
7. Reference internship report supplied for formatting and organizational guidance.